Is a Phone Enough? Measuring an iPhone as a Ground Robot's Only Sensor Head and Computer

Yojan Gautam
[TODO: affiliation, city, country]
[TODO: email]

***Abstract*—Consumer phones carry cameras, a LiDAR scanner, an IMU, GNSS, a neural accelerator and a battery, which makes them an attractive single board for cheap ground robots. Earlier phone robots show that the idea works, but none reports whether the phone's own estimates and sensor timing meet the numbers a control loop needs, with every subsystem running at once on a moving vehicle. We run Terra, a skid-steer rover whose only sensor head and computer is an iPhone with LiDAR, and contribute three things. First, we turn the rover's control, mapping, detection and guidance stack into nine measurable requirements on the phone. Second, we compare ARKit visual–inertial odometry (VIO) with OptiTrack motion capture over 17 straight runs. VIO speed noise, bias, delay and heading meet their requirements, but its speed scale reads 0.912 of truth on average (s.d. 0.065, 16 runs, one further run at 0.60). Third, we introduce a per-hop timing probe that attributes every stale control tick to iOS or to the app. iOS delivered IMU samples at 99–100 Hz with a 99th-percentile delay of at most 2.3 ms in every run, and all 233 stale ticks in a 336 s run came from an unoptimised Debug build of the app's own depth-to-map step; the Release build had none. We close with the requirements still open: depth, GNSS and 90-minute endurance.**

***Index Terms*—smartphone robotics, visual–inertial odometry, sensor timing, low-cost mobile robots, field robotics**

I. Introduction

A fleet of cheap ground robots covers more ground than one expensive platform, provided each robot is cheap to build, lose and replace. The dedicated computer and sensor suite are a large part of what makes a robot expensive to field: each needs power and thermal design, a machine image and its own bring-up. A current phone already holds most of that suite in one sealed, mass-produced package, and software reaches it as an app install. This paper asks whether that package is good enough to be the robot's *only* sensor head and computer.

Phones have driven robots before [1]–[3]. What those works do not report is whether the phone's own pose estimate and sensor timing meet the specific numbers a closed control loop needs, measured on the vehicle, with mapping, detection and video running on the same chip. Phone VIO, LiDAR, IMU and GNSS have each been benchmarked, but one at a time, hand-held or static, on an otherwise idle phone (Section II).

We study this question on Terra, a skid-steer rover whose sensing, state estimation, mapping, object detection, planning and speed control all run on an iPhone (Fig. 1). Our contributions are:

1) **A requirements framework.** Nine requirements R1–R9 on the phone, each derived from the part of the robot that consumes the sensor, with a threshold that a measurement can pass or fail (Section IV).
2) **VIO against motion capture on a ground robot without wheel encoders.** Seventeen straight runs scored against R1–R4 (Section V).
3) **A per-hop sensor timing method** that stamps each sample from sensor to control core and attributes every stale control tick to iOS or to the app. Applied to iOS under full robot load, it shows that the IMU stalls we saw in the field came from a Debug build, not from iOS (Section VI).

Section VII scores the phone against all nine requirements and lists the experiments that remain.

II. Related Work

Phones as robot computers. Oros and Krichmar built a teaching robot around an Android phone for about $350 excluding the phone [1]. NASA's Smart SPHERES added a smartphone to free-flyers aboard the International Space Station [2]. OpenBot, the closest predecessor to Terra, drives a $50 body with an Android phone and runs person following and learned navigation, with its detector at 4 to 33 frames per second depending on the phone [3]. None of these reports the accuracy of the phone's pose estimate on the robot or the timing of its sensors, and none uses an iPhone, ARKit or LiDAR.

VIO on phones. ADVIO compared ARCore, ARKit and Tango with academic VIO on hand-held sequences [4], following benchmarks on flying-robot hardware [5]. Feigl et al. found scale errors of up to 14.4 cm per metre for phone and headset VIO on a factory floor [6]. Kim et al. found ARKit the most accurate of four commercial systems against OptiTrack, but with the phone hand-held or on a car seat, and did not quantify scale [7]. Wu et al. proved that VIO on a ground vehicle moving in straight lines or arcs gains unobservable directions, scale among them, and restored them with wheel encoders [8]. Terra drives straight lines and arcs and has no encoders.

Phone depth, IMU and GNSS. iPhone and iPad LiDAR has been characterised on static scenes from a hand-held device [9]–[11]. Phone IMUs sample at irregular, device-specific rates [12] and have been characterised for bias and noise on a stationary phone [13]; camera–IMU time offsets degrade VIO [14]. Phone GNSS has median errors of 5–8.5 m static [15] and much larger near buildings [16]. On-

device inference is benchmarked on idle phones [17], [18], and sustained load drives phones into thermal throttling [19].

Gap. To our knowledge no work reports, for iOS, the delay from IMU sample to app and its worst-case gaps under a concurrent ARKit and LiDAR load, nor ARKit's speed scale on a ground robot against motion capture.

III. Platform

Rover and phone. The rover is CrabTrapBot, a skid-steer chassis without wheel encoders. An iPhone with a LiDAR scanner (hardware identifier iPhone17,2, iPhone 16 Pro Max, iOS 27.0.1) is mounted in landscape at a camera height of 0.23 m. A Raspberry Pi receives wheel commands over Bluetooth Low Energy at 20 Hz and drives the motors; it holds no autonomy and stops the motors if commands stop arriving. [TODO: chassis dimensions, mass, motor and battery spec, and bill of materials with and without the phone.]

Software. TerraPhone, a Swift app, gathers sensor data, builds a 0.1 m occupancy grid from LiDAR depth, runs a YOLO bucket detector on the Neural Engine and streams video to the operator station. The Rust core runs the state estimator, planner, safety gates and a 100 Hz speed loop. The estimator takes the latest ARKit VIO pose and velocity and replays the IMU samples retained since, giving forward speed $\hat{v}$ and gyro yaw rate $\hat{\omega}$. It withholds the estimate, and the controller stops, if VIO reports lost tracking, if the newest IMU sample is older than 0.1 s, or if VIO is older than 0.35 s. Most phone-side work shares one serial dispatch queue, `terra.control` (Fig. 1): a long block there delays every IMU sample behind it. The Rust core is always compiled with optimisation; the Swift app is not, in a Debug build.

IV. Requirements

"Good enough" is testable only against numbers. Table I derives one requirement for each phone capability from the component of Terra that consumes it. Where a threshold is a constant in Terra, the table gives its source; where we propose it, the table says so.

This paper measures R1–R5 and reports the protocols for R6–R9, which are pending (Section VII). R5 is the one that most directly decides whether a phone can host a control loop at all, and the one the literature says least about.

TABLE I
Requirements on the phone. "Proposed" marks thresholds set in this paper; the rest are constants of the robot.

ID	Capability	Requirement	Source
R1	VIO speed scale	Steady speed within 0.97–1.03 of truth; bias at rest $\leq$ 0.01 m/s	Speed-loop accuracy
R2	VIO speed noise	$\leq$ 0.02 m/s in the control band	Speed loop
R3	VIO delay	Below the loop's delay margin, 382 ms at the tested gains	Speed-loop stability
R4	Heading	Error $< 30°$ to admit an outdoor mission	Heading gate
R5	IMU delivery	Newest IMU sample younger than 0.1 s at every control tick	Estimator stale gate
R6	Depth	0.15 m obstacles seen at $\geq$ 3 m; range error $<$ one 0.1 m cell; no false obstacles on bare floor	Map resolution; range proposed
R7	Detection	Recall $\geq$ 0.9 within 10 m above the 0.8 confirm threshold	Target confirm rule; recall proposed
R8	GNSS	95% horizontal error $\leq$ 10 m	Proposed, from R7's range
R9	Endurance	90 min of full load without the *critical* thermal state, with battery left	Competition match length

TABLE II
VIO against OptiTrack, scored against R1–R4.

Req.	Threshold	Measured	Verdict
R1 scale	0.97–1.03	Mean 0.912, s.d. 0.065 (16 runs); one run 0.60; all 17: mean 0.894, median 0.92	Fail
R1 bias	$\leq$ 0.01 m/s	0.0009 m/s, s.d. 0.003 m/s at rest	Pass
R2 noise	$\leq$ 0.02 m/s	0.007 m/s in band, 0.012 m/s above	Pass
R3 delay	$<$ 382 ms	About 50–70 ms relative to the gyro	Pass
R4 heading	$< 30°$	Within 0.1° of OptiTrack over a 475° arc	Pass

V. VIO Against Motion Capture

A. Method

The rover was driven in an OptiTrack volume of about 12×8 m on October 26 and 27, 2026. OptiTrack (Motive 3, NatNet 4) tracked a rigid body on the chassis at 120 Hz; over a 3 m tape it read within 0.01 m, so we take it as ground truth. A bridge republished the mocap pose on the robot's message bus, and one laptop program recorded both poses and drove the rover straight at a set speed and distance through teleoperation. The phone's own telemetry journal was copied off the phone after each session. For each run we compare the steady part of the phone's forward speed, after the speed-up and before the stop, with OptiTrack's (Fig. 2). The mocap rigid body sits 0.17 m from the phone's reference point along the forward axis, so on straight runs the lever arm adds no forward speed.

B. Results

Seventeen straight runs at 0.2–0.4 m/s were usable (Fig. 3). Table II scores them against R1–R4.

ARKit gives the speed loop a quiet, unbiased and fast signal with an accurate heading, but its speed scale is about 9% short and varies from run to run. Because the loop regulates the phone's estimate, the rover drives faster than commanded by the same factor (Fig. 2). Two further observations bound the cause. When the rover was carried by hand over 4.42 m (OptiTrack), the phone read 4.06 m, a ratio of 0.92, so the wheels and drive vibration are not the cause. On one arc, the VIO position jumped by 2.75 m, which a speed loop that differentiates position would see as a spike. The R3 delay is coarse because the journal sampled the estimate at about 8 Hz.

Two explanations remain. The first is the room: OptiTrack's infrared strobes may disturb the LiDAR, and the floor is plain. The second is the motion: Wu et al. predict weak scale observability for exactly this straight-line ground motion [8]. If

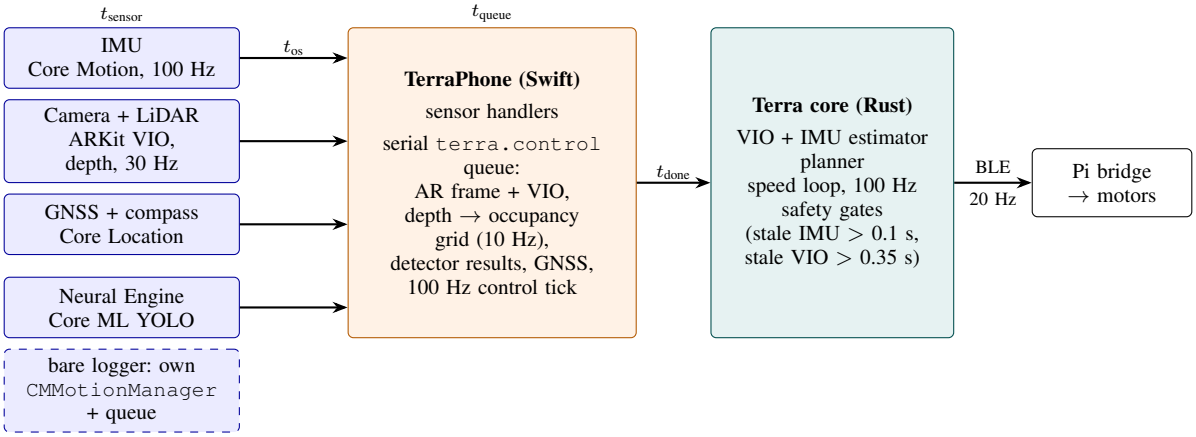


Fig. 1. Terra's sensing and control pipeline on the phone. Every sample is stamped when the sensor produced it (t_{sensor}), when iOS handed it to the app (t_{os}), when its block started on the serial control queue (t_{queue}), and when it reached the Rust core (t_{done}). iOS delivery is $t_{os} - t_{sensor}$ and queue wait is $t_{queue} - t_{os}$. A separate bare logger (dashed) reads Core Motion on its own queue, giving an iOS-only baseline with none of the app's code in the way.

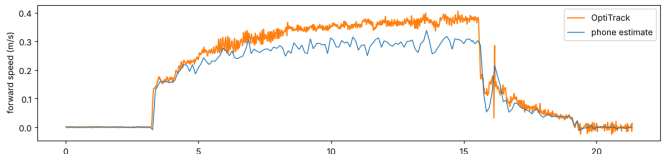


Fig. 2. One straight run at a 0.3 m/s target (session 1, run 2): forward speed from OptiTrack (orange) and from the phone's estimate (blue) against time in seconds. The speed loop holds the phone's estimate near 0.3 m/s while the rover's true speed settles near 0.37 m/s; the steady ratio for this run is 0.81. The phone trace is sparser because the journal recorded it at about 8 Hz.

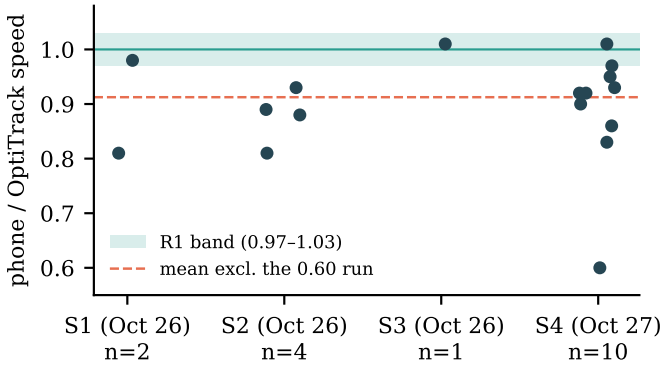


Fig. 3. Steady forward speed, phone VIO divided by OptiTrack, for each of the 17 straight runs, by session. Only 4 of the 17 runs fall in the R1 band.

the room is the cause, the ratio should return to 1 elsewhere; if the motion is, it should stay below 1 in any room and recover only with more varied motion.

C. Pending: scale away from the mocap room (E1b)

Ground truth is a tape: marks 10.00 m apart, the stop position marked by hand. Factors are place (mocap room with strobes on, strobes off, textured hallway, plain hallway, outdoor turf), motion (straight, straight with a 1 m side-step weave) and speed (0.3 m/s everywhere; 0.1 and 0.5 m/s in the textured hallway). With the measured spread of 0.065, 11 runs per cell detect a 0.08 difference between places with 80% power at $\alpha = 0.05$; detecting 0.05 needs 27. If one place-independent ratio fits within $\pm 3\%$, a fixed scale correction passes R1; otherwise scale needs an independent reference such as GNSS speed outdoors. [TODO: Run E1b (120 runs); add a two-way ANOVA table of ratio on place and motion, and replace this subsection with results.] [TODO: Run E1c: yaw rate in arcs and turns against OptiTrack at 100 Hz logging, after the landscape-mount gyro fix.]

VI. Sensor Timing Under Load

A. Motivation

During the mocap sessions and earlier test drives the rover held repeatedly with a stale-IMU fault: at some control tick the newest IMU sample was older than 0.1 s. A phone's sensor timing is set by the operating system, which the app cannot see into. The question for R5 is therefore whether the stale samples come from iOS, so that a phone cannot host the loop, or from the app, so that it can be fixed.

B. Per-hop timing probe

The probe stamps every IMU, AR, detector, GNSS and heading sample at the four points of Fig. 1 and writes them, with every control tick and every block that ran on the control queue, to a CSV file on the phone. The *iOS delivery* delay is $t_{os} - t_{sensor}$; the *queue wait* is $t_{queue} - t_{os}$. A bare logger, a second Core Motion manager on its own queue, records what iOS delivers with none of Terra's code in the path.

At each control tick the IMU age is $t_{tick} - t_{sensor}$ of the newest sample pushed to the Rust core. When the age reaches the gate, the analysis looks at the *next* IMU sample after the newest one pushed, and assigns the stale tick exactly one cause (Fig. 4): `ios_sensor_gap` if that sample had not yet been taken at the tick; `ios_delivery` if it had been taken but iOS had not yet delivered it; `terra_queue` if iOS had delivered it and it was waiting on the control queue, in which case the block running on the queue is named as the blocker.

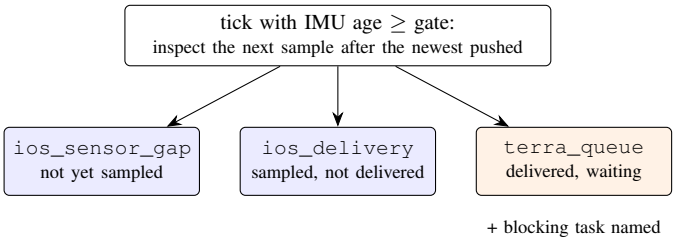


Fig. 4. Attribution of a stale control tick. Blue causes are iOS; orange is the app.

TABLE III
SENSOR TIMING WITH TERRA'S FULL LOAD, ROVER STILL. STALE TICKS ARE CONTROL TICKS WHOSE NEWEST IMU SAMPLE WAS OLDER THAN THE GATE. DETECTOR LATENCY IS $t_{\text{DONE}} - t_{\text{SENSOR}}$. THERMAL STATE REACHED *serious* IN EVERY RUN.

Build	C2* Debug	C2/C3 Release	C2/C3 Release	C4 Release
Duration (s)	336	122	52	38
IMU rate (Hz)	99.1	99.4	99.4	99.4
iOS delivery p50/p99 (ms)	0.9/2.1	1.0/2.0	1.0/2.1	1.1/2.3
Queue wait p99 (ms)	2414	8.7	14.5	20.3
Control queue busy (%)	60	7	10	12
Depth-to-map p50/max (ms)	65.7/114.3	9.4/21.9	11.8/29.5	15.6/28.9
Control tick rate (Hz; 100)	39.0	99.8	98.4	95.6
IMU age p99/max (ms)	104/2308	14/28	19/37	24/32
Stale ticks, 0.1 s gate	233 (1.8%)	0	0	0
Stale ticks, 0.5 s gate	1	0	0	0
AR frame rate (Hz)	27.2	32.1	30.0	30.0
Detector rate (Hz)	4.5	4.3	4.3	4.3
Detector p50/p99 (ms)	108/172	71/80	76/91	78/97

C. Conditions

All runs used the phone of Section III on October 27, 2026, with the rover still. A first short capture gave a pilot for two iOS-only conditions: bare IMU (C0, about 10 s) and IMU with ARKit, LiDAR depth and meshing running (C1, about 7 s). Four runs then used Terra's full load: tracking on, the detector running (it always runs while tracking, so the *tracking* and *tracking + detector* conditions coincide), and the operator station connected; one Release run added FPV video (C4). The Debug run lasted 336 s, during which the autonomy level was changed, so we mark it C2*. The Release runs lasted 122, 52 and 38 s, and were recorded after the Debug run with the phone already in the *serious* thermal state. Debug and Release differ only in the Swift optimiser; iOS and the Rust core are identical.

D. Results

iOS is not the cause. In every run iOS delivered IMU samples at 99–100 Hz with a 99th-percentile delay between 2.0 and 2.3 ms (Table III, Fig. 5), which leaves almost all of the 0.1 s budget to the app. In the pilot, the bare logger saw p50/p99 delivery of 3.4/4.1 ms with IMU alone (C0) and 1.1/3.0 ms with ARKit, LiDAR and meshing running (C1). iOS stalled twice in all the data. It held samples for 1.17 s while an ARKit session started (113 samples held, then delivered in a burst), so a rover must not be allowed to drive

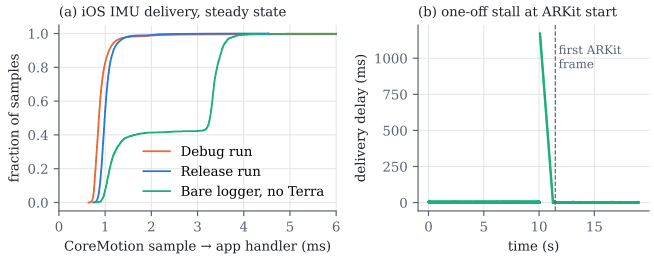


Fig. 5. iOS IMU delivery, Core Motion sample to app handler. (a) In steady state, delivery is under about 2 ms in the Debug and Release runs and under 5 ms for the bare logger. (b) The one start-up stall: iOS held samples for 1.17 s while the ARKit session started.

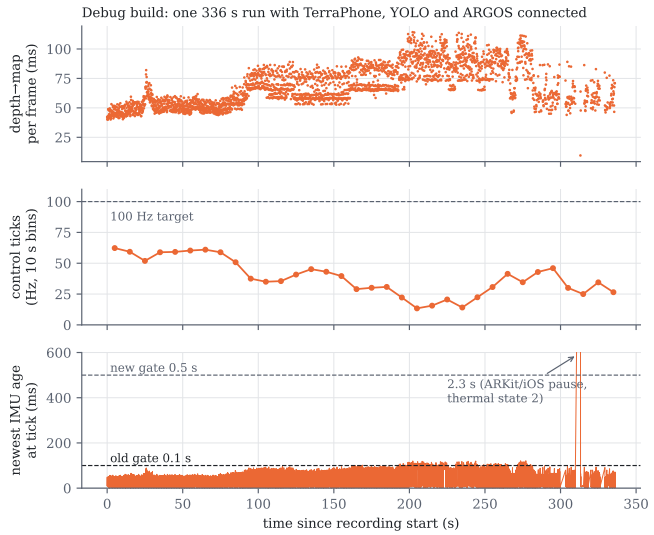


Fig. 6. The Debug run (C2*, 336 s). Top: time the Swift depth-to-map step held the control queue per frame. Middle: control ticks per second, 10 s bins. Bottom: IMU age at each tick, with the 0.1 s gate and a looser 0.5 s gate.

until tracking has run for a few seconds. It also paused the IMU for 2.3 s at 311 s into the Debug run, with the phone in the *serious* thermal state and AR frames gapping by up to 3.8 s at the same time; its cause is unknown.

The Debug build is the cause. All 233 stale ticks of the Debug run were `terra_queue`, and in each one the blocker was the Swift depth-to-map step of an AR frame. That step grew from about 45 ms per frame to over 100 ms during the run (Fig. 6). At 10 Hz it kept the control queue 60% busy, and the 100 Hz control loop averaged 39 Hz. IMU samples reached the app within about 2 ms and then waited behind the map step. In Release, the same step took 9–16 ms (p50) and never more than 30 ms, the queue was 7–12% busy, and the loop ran at 96–100 Hz without a stale tick (Fig. 7). The Release runs started with the phone already hot, so heat alone does not explain the Debug slowdown.

Loosening the gate to 0.5 s would have left one stale tick in the Debug run, the 2.3 s iOS pause, instead of 233. A looser gate hides queue delays rather than removing them, so we take the Release numbers, not the gate, as the evidence for R5. The

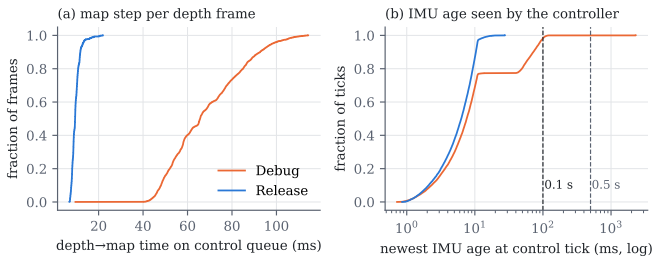


Fig. 7. Debug against Release, empirical CDFs. (a) Depth-to-map time per frame. (b) IMU age at each control tick (log scale) with the 0.1 s and 0.5 s gates.

TABLE IV
IS THE PHONE ENOUGH? STATUS OF EACH REQUIREMENT.

Req.	Verdict	Basis
R1 scale	Fail as measured	0.91 in the mocap room; E1b decides if a fixed correction suffices
R2 noise	Pass	Sec. V
R3 delay	Pass	Sec. V (coarse, 8 Hz journal)
R4 heading	Pass (VIO)	Compass and GNSS course pending
R5 IMU	Pass in Release; fail in Debug	iOS p99 $\leq$ 2.3 ms; Release 0 stale ticks in 212 s; two iOS stalls (1.17 s, 2.3 s); static and short
R6 depth	Pending	[TODO: E3: targets at 0.5–5 m, four lighting conditions incl. mocap strobes and noon sun, parked and driving]
R7 detection	Rate measured; accuracy pending	4.3 Hz, 71/80 ms p50/p99 under full load (Release). [TODO: E4: real buckets at 2–12 m, sun and shade]
R8 GNSS	Pending	[TODO: E5: static CEP50/CEP95 on four surveyed points, 30 m driving runs, compass near motors]
R9 endurance	Pending	Serious thermal state after about 4 min in C2* (Debug, still). [TODO: E6: 90 min full load, bench and field, battery and thermal logged]

broader lesson for phone robotics is methodological: an app's Debug build can turn a sound pipeline into one that looks like the operating system is failing, and only a per-hop probe tells the two apart.

E. Limitations

The three Release runs total 212 s, none longer than the roughly 150 s the Debug slowdown took to appear; all runs were static; and only one phone model was tested. [TODO: Run a Release run of at least 10 minutes with full load and report whether the late iOS IMU and AR stalls reappear; extend Table III with it.] [TODO: Run C5: the same probe while driving straight lines at the waypoint and planner autonomy levels.]

VII. VERDICT AND OPEN REQUIREMENTS

Table IV collects the status of all nine requirements. The parts of the phone that the inner control loop depends on most, VIO noise, delay and heading and iOS's IMU delivery, meet their requirements with margin. The stale-IMU holds that first suggested otherwise came from the app's own unoptimised code on a shared queue and disappeared in Release. The VIO scale error is real and repeatable in the mocap room; it is small enough that the loop stays stable and the rover stays consistent with its own map, and it matters for absolute distances. What remains open is the hardest part of the claim: whether all of this holds while driving, outdoors, with every subsystem on for a 90-minute match. [TODO: Fill R6–R9 from experiments E3–E6 and update the abstract and this paragraph.]

VIII. CONCLUSION

We stated what a phone must deliver to be a ground robot's only sensor head and computer as nine testable requirements, and measured the first five on an iPhone driving a skid-steer rover. ARKit VIO is quiet, unbiased, fast and accurate in heading, but under-reads speed by about 9% in our mocap room. iOS delivers IMU samples within about 2 ms under full ARKit, LiDAR, detection and video load; the stalls we first attributed to the phone came from a Debug build, which our per-hop probe exposed. The probe and analysis scripts are part of the Terra code base. [TODO: State the code and data release (repository URL, licence) once decided.]

ACKNOWLEDGMENT

[TODO: Advisor, lab, motion-capture facility, funding.]

REFERENCES

[1] N. Oros and J. L. Krichmar, "Smartphone based robotics: Powerful, flexible and inexpensive robots for hobbyists, educators, students and researchers," Center for Embedded Computer Systems, University of California, Irvine, Tech. Rep. CECS TR 13-16, 2013.

[2] T. Fong, M. J. Micire, T. Morse, E. Park, C. Provencher, V. To, D. W. Wheeler, D. Mittman, R. J. Torres, and E. Smith, "Smart SPHERES: A telerobotic free-flyer for intravehicular activities in space," in *AIAA SPACE 2013 Conference and Exposition*, 2013, aIAA 2013-5338.

[3] M. Müller and V. Koltun, "OpenBot: Turning smartphones into robots," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021, arXiv:2008.10631.

[4] S. Cortés, A. Solin, E. Rahtu, and J. Kannala, "ADVIO: An authentic dataset for visual-inertial odometry," in *European Conference on Computer Vision (ECCV)*, 2018, pp. 419–434.

[5] J. Delmerico and D. Scaramuzza, "A benchmark comparison of monocular visual-inertial odometry algorithms for flying robots," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[6] T. Feigl, A. Porada, S. Steiner, C. Löffler, C. Mutschler, and M. Philippsen, "Localization limitations of ARCore, ARKit, and Hololens in dynamic large-scale industry environments," in *Proceedings of the 15th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP), Volume 1: GRAPP*, 2020, pp. 307–318.

[7] P. Kim, J. Kim, M. Song, Y. Lee, M. Jung, and H.-G. Kim, "A benchmark comparison of four off-the-shelf proprietary visual–inertial odometry systems," *Sensors*, vol. 22, no. 24, p. 9873, 2022.

[8] K. J. Wu, C. X. Guo, G. Georgiou, and S. I. Roumeliotis, "VINS on wheels," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 5155–5162.

[9] G. Luetzenburg, A. Kroon, and A. A. Bjørk, "Evaluation of the Apple iPhone 12 Pro LiDAR for an application in geosciences," *Scientific Reports*, vol. 11, p. 22221, 2021.

[10] M. Vogt, A. Rips, and C. Emmelmann, "Comparison of iPad Pro's LiDAR and TrueDepth capabilities with an industrial 3D scanning solution," *Technologies*, vol. 9, no. 2, p. 25, 2021.

[11] G. Baruch, Z. Chen, A. Dehghan, T. Dimry, Y. Feigin, P. Fu, T. Gebauer, B. Joffe, D. Kurz, A. Schwartz, and E. Shulman, "ARKitScenes: A diverse real-world dataset for 3D indoor scene understanding using mobile RGB-D data," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021, arXiv:2111.08897.

[12] A. Stisen, H. Blunck, S. Bhattacharya, T. S. Prentow, M. B. Kjærgaard, A. Dey, T. Sonne, and M. M. Jensen, "Smart devices are different: Assessing and mitigating mobile sensing heterogeneities for activity recognition," in *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2015.

[13] A. Kos, S. Tomažič, and A. Umek, "Evaluation of smartphone inertial sensor performance for cross-platform mobile applications," *Sensors*, vol. 16, no. 4, p. 477, 2016.

[14] M. Li and A. I. Mourikis, "Online temporal calibration for camera–IMU systems: Theory and algorithms," *The International Journal of Robotics Research*, vol. 33, no. 7, pp. 947–964, 2014.

[15] P. A. Zandbergen and S. J. Barbeau, "Positional accuracy of assisted GPS data from high-sensitivity GPS-enabled mobile phones," *The Journal of Navigation*, vol. 64, no. 3, pp. 381–399, 2011.

[16] K. Merry and P. Bettinger, "Smartphone GPS accuracy study in an urban environment," *PLoS ONE*, vol. 14, no. 7, p. e0219890, 2019.

[17] A. Ignatov, R. Timofte, A. Kulik, S. Yang, K. Wang, F. Baum, M. Wu, L. Xu, and L. Van Gool, "AI benchmark: All about deep learning on smartphones in 2019," in *IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2019, pp. 3617–3635.

[18] V. J. Reddi, D. Kanter, P. Mattson *et al.*, "MLPerf mobile inference benchmark: An industry-standard open-source machine learning benchmark for on-device AI," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 4, 2022, pp. 352–369.

[19] J. Xiao, Q. Huang, X. Chen, and C. Tian, "Understanding large language models in your pockets: Performance study on COTS mobile devices," arXiv:2410.03613, 2024, preprint.